intertrust®

Getting Started with ExpressPlay Tutorial

Copyright © 2020 Intertrust Technologies. All Rights Reserved.

PDF Viewer **3**

Lesson1 - About Digital Rights Management **3**
Mainstream DRM systems supported by ExpressPlay 3

Lesson 2 - Setting up ExpressPlay Services **4**
ExpressPlay product lines 4
Registering for ZenDesk ticket system 4
Creating an API Key 4
Configuring PlayReady DRM and FairPlay DRM 5

Lesson 3 - Content Packaging **6**
Packaging DASH content using Bento4 6
Packaging CMAF content using Bento4 7
Packaging HLS content using Bento4 8
Packaging content for Marlin 9

Lesson 4 - DRM License redemption workflow **10**
Token Lifetime 11
The ExpressPlay REST API 11
Generating ExpressPlay Tokens 11
FairPlay Streaming 12
Widevine Modular Token Generation 12
PlayReady DRM Token Generation 13
Marlin MS3 and Marlin Broadband Token Generation 14

Lesson 5 - The ExpressPlay Audit Log **14**

Lesson 6 - Content Playback **14**
Shaka Player online DRM demo 15
HTML5 Playback 16
Playback using ExoPlayer 16
Playback using AVPlayer 16
Playback of Marlin protected content 17

Lesson 7 - The Key Management Service (KMS) **17**
Third-Party Packaging Services 18
KMS Sample commands 19
Retrieving a Content Key 19
Generating a Content Key 20
Storing a content key under a specific Key identifier 20
Updating a content key 20

Implementing Key Rotation 21

Lesson 8 - License Proxy Deployment Architecture 22

Performance considerations 23

Payload and HTTP headers 23

PDF Viewer

This document provides a series of lessons introducing ExpressPlay to the reader.

The document is best viewed in the Adobe Acrobat PDF viewer available for free from the Adobe web site. Other pdf viewers such as the one included with the Chrome browser reformat the command line code snippets included in this document, resulting in an error when the snippets are copy pasted to a terminal window.

Lesson1 - About Digital Rights Management

Digital rights management (DRM) technology enables content owners to deliver digitally encoded audio-visual content, while preventing unauthorized redistribution of the content.

The technology has several key components:

1. **Content Encryption** - The content is encrypted using strong encryption technologies, such as AES 128, that require the playback device to obtain a decryption key in order to render the content. As the content is encrypted it can freely be distributed in the most efficient means possible (e.g. via CDNs with no authentication) without having to worry about unauthorized viewing.
2. **DRM License** - A DRM license is typically a small file that contains the content decryption key (in encrypted form) along with some rules/policies on how the content is to be used. Typical rules set in a license define the output controls (e.g. HDCP requirements) and /or license validity period. Like the content itself, the license can be delivered freely and openly as the key itself is encrypted and the entire file is signed preventing any modifications to the specified rights.
3. **Secure Client** - A secure client is responsible for evaluating the rules in the license. The client is hardened via software and/or hardware against tampering and is responsible for ensuring the content decryption key is released only to trusted playback systems on the device. The type of hardening of the client determines the level of security of the client hence the type of content that a device can consume All major browsers and operating systems ship with a pre-integrated trusted client.

Mainstream DRM systems supported by ExpressPlay

The following DRM systems have been approved by all the major content owners and are supported by the ExpressPlay Service.

- **Marlin DRM** - An open standards-based DRM system developed by major CE manufacturers and Intertrust Technology. Intertrust provides Marlin DRM client SDKs for

smart TVs, set-top boxes, desktops, and mobile systems. Further information on Marlin DRM is available here

- **Widevine Modular** - Google's DRM technology that is pre-integrated with Google Chrome, Chromium, Android mobile, set-top boxes, and TVs. Further background on Widevine Modular is available here. (The previous generation of Widevine technology called Widevine Classic is deprecated and not supported by ExpressPlay)
- **FairPlay Streaming -** Apple's DRM technology that is pre-integrated with Safari and iOS devices. Further information on FairPlay Streaming is available here
- **Microsoft PlayReady** - Microsoft's DRM technology that is pre-integrated with Microsoft browsers, operating systems as well as some smart TVs and set-top boxes. Further information on PlayReady is available here

Lesson 2 - Setting up ExpressPlay Services

ExpressPlay product lines

ExpressPlay offers the following product lines:

1. A multi-DRM license service that supports all the major DRM systems
2. Client SDKs implementing Marlin DRM client on a wide range of devices
3. Session-based watermarking solutions

Use of ExpressPlay requires establishing an account with the ExpressPlay service, a free 30-day trial is granted on sign up.

While not strictly required for this tutorial you can sign up for a free trial here.

Registering for ZenDesk ticket system

The ExpressPlay ZenDesk is used to submit support tickets as well as a comprehensive FAQ.

Registration for the ExpressPlay ZenDesk ticket system is done here

Help Center

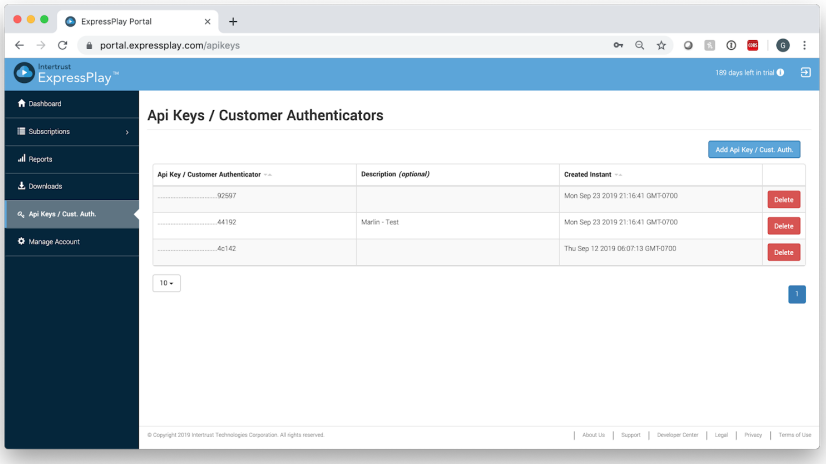
Once registered for the ExpressPlay ZenDesk you will have access to the ExpressPlay help center, available here

Creating an API Key

After an account is created, the next step is to create an API key here.

An API key is required to call the ExpressPlay REST APIs. Some important things to note, regarding API keys:

- For legacy reasons, this API key is also known as a Customer Authenticator. In all the calls to the ExpressPlay API, the key is specified in the customerAuthenticator parameter.
- The API Key is highly confidential, anyone with access to the key can issue DRM licenses on the OTT service's behalf.
- ExpressPlay only stores an MD5 hash of the API key, hence it is not possible to retrieve the key once created through the console UX. So please take note of the key.
- It is possible to create and deprecate API keys dynamically. This is particularly important in case an API key is compromised.
- It is also possible to use multiple API keys concurrently, this allows for seamless replacement of API keys with no disruption to services that call ExpressPlay.



API Key management screen

Configuring PlayReady DRM and FairPlay DRM

Once registered, you will have access to the Marlin and Widevine DRM license services. Configuration of PlayReady and FairPlay Streaming requires additional steps.

Subscription to PlayReady and FairPlay Streaming DRMs is done through the subscriptions section in the account management UX, available here (you will need to sign in first).

1. Microsoft requires some basic information from each PlayReady deployment, which you will be prompted to enter once you enable the PlayReady subscription.

2. Apple requires an application through the Apple developer portal. Detailed instructions on enabling FairPlay Streaming support are available here

Lesson 3 - Content Packaging

The dominant streaming formats used by OTT services are Apple HLS,MPEG DASH. The more recently specified CMAF (Common Media Application Format, as defined in ISO/IEC 23000-19) protocol is also widely supported on recent platforms.

Preparing content for either standard involves:

- Encoding software/services that are used to transcode the original content to multiple resolutions, bitrates and display sizes
- Packaging software/services that adapt the media to the streaming format (e.g. generate the manifest describing the media) and encrypt it

ExpressPlay does not provide content transcoding or packaging services, thus third-party tools/services are required. Several of these tools/services have been integrated with the ExpressPlay key management service to help simplify the content packaging workflow (for further details please see Section 7 - The Key Management Service).

Packaging DASH content using Bento4

We will demonstrate packaging content in DASH format using the open-source bento4 toolchain (available here).

Once you have downloaded Bento4 follow these steps to prepare the sample content

1. Download the sample content here or use the curl command below

```
curl -O
https://expressplay.s3.amazonaws.com/sample%20content/Sample%20A/tears-of-steal-teaser-1280-720.mp4
```

2. Fragment the content using the mp4fragment tool as demonstrated below (a necessary step before it can be packaged)

```
mp4fragment tears-of-steal-teaser-1280-720.mp4 \
tears-of-steal-teaser-1280-720.fmp4
```

3. Use the mp4dash command to create DASH content that is encrypted and contains signaling for Marlin, Widevine and PlayReady DRMs.

```
mp4dash --o output_dash \
--encryption-key=33333333333333333333333300000000:7f45050639469c1fb
2a85db718c28ed7 \
--marlin --widevine
--widevine-header=provider:intertrust#content_id:2a \
--playready
--playready-header=https://pr.service.expressplay.com/playready/Rig
htsManager.asmx \
tears-of-steal-teaser-1280-720.fmp4
```

After executing the command an output_dash folder will be created containing the encrypted content

When requesting the license for this content you will need to provide the Content Key and the Key Identifier that were specified:

1. The content-encryption key specified is 7f45050639469c1fb2a85db718c28ed7
2. The key identifier specified is 33333333333333333333333300000000

Packaging CMAF content using Bento4

Very similarly as packaging content in DASH format using the open-source bento4 toolchain, it is also possible to package content in CMAF format using the open-source bento4 toolchain (available here).

1. Download the sample content here or use the curl command below

```
curl -O
https://expressplay.s3.amazonaws.com/sample%20content/Sample%20A/te
ars-of-steal-teaser-1280-720.mp4
```

2. Fragment the content using the mp4fragment tool as demonstrated below (a necessary step before it can be packaged)

```
mp4fragment tears-of-steal-teaser-1280-720.mp4 \
tears-of-steal-teaser-1280-720.fmp4
```

3. Use the mp4dash command to create CMAF content that is encrypted and contains signaling for Apple FairPlay Streaming, Marlin, Widevine and PlayReady DRMs. (where applicable)

```
mp4dash --o output_cmaf \
--encryption-key=33333333333333333333333300000000:7f45050639469c1fb
2a85db718c28ed7:00000000000000000000000000000000 \
--encryption-cenc-scheme=cbcs \
--hls \
--marlin \
--fairplay-key-uri=skd://33333333333333333333333300000000 \
--widevine \
--widevine-header=provider:intertrust#content_id:2a \
--playready \
--playready-version=4.3 \
--playready-header=https://pr.service.expressplay.com/playready/Rig
htsManager.asmx \
tears-of-steal-teaser-1280-720.fmp4
```

After executing the command an output_cmaf folder will be created containing the encrypted content

Note: The output_cmaf folder will contain several media manifests (with different signaling) for the same encrypted content

- a DASH manifest with Marlin, Widevine and PlayReady DRMs
- A HLS playlist with FairPlay Streaming, Widevine and PlayReady DRMs

When requesting the license for this content you will need to provide the Content Key and the Key Identifier that were specified:
1. The content-encryption key specified is 7f45050639469c1fb2a85db718c28ed7
2. The key identifier specified is 33333333333333333333333300000000
When requesting a FairPlay Streaming license you will need to specify as well an Initialization Vector (IV) parameter:
3. The IV specified is 00000000000000000000000000000000

Packaging HLS content using Bento4

We will demonstrate packaging of content in HLS format using the open-source bento4 toolchain (available here)

Once you have downloaded Bento4 follow these steps to prepare the sample content

1. Download the sample content here or use the curl command below

```
curl -O
https://expressplay.s3.amazonaws.com/sample%20content/Sample%20A/te
ars-of-steal-teaser-1280-720.mp4
```

2. Package the content using the command below. When requesting a FairPlay Streaming license you will need to specify a content key, a key identifier as well as an Initialization Vector (IV). In the example below:
   a. The content-encryption key is 7f45050639469c1fb2a85db718c28ed7
   b. The key identifier is 33333333333333333333333300000000
   c. The IV is 00000000000000000000000000000000
3. After executing the command an output_hls folder will be created containing the encrypted content.

```
mp4hls --output-dir  output_hls --output-single-file \
--encryption-mode=SAMPLE-AES \
--encryption-key=7f45050639469c1fb2a85db718c28ed7000000000000000000
00000000000000 \
 --encryption-iv-mode=fps
--encryption-key-format=com.apple.streamingkeydelivery \
--encryption-key-uri=skd://33333333333333333333333300000000 \
tears-of-steal-teaser-1280-720.mp4
```

When requesting a FairPlay Streaming license you will need to specify a content key, a key identifier as well as an Initialization Vector (IV). In the example above::

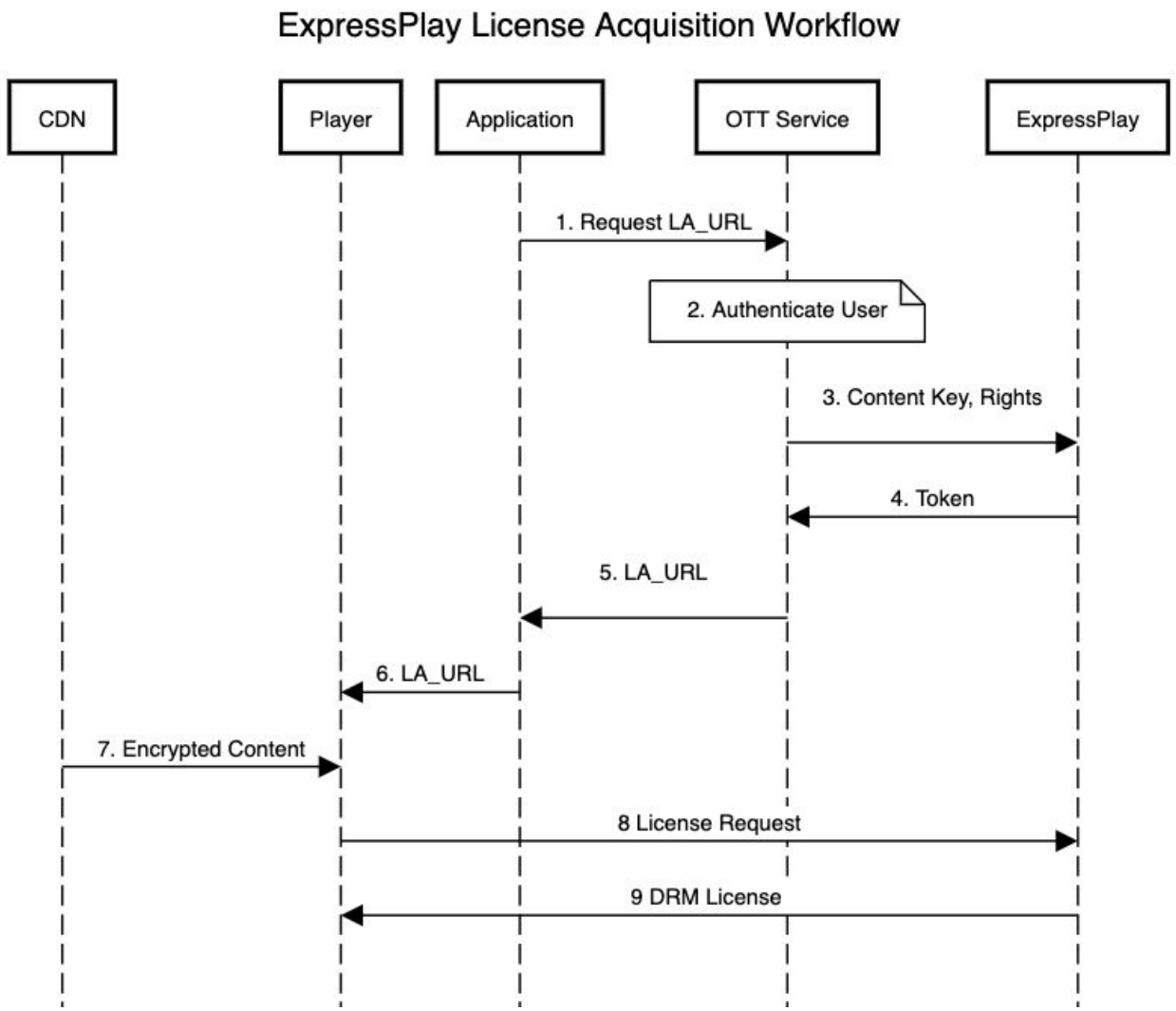
1. The content-encryption key is 7f45050639469c1fb2a85db718c28ed7
2. The key identifier is 33333333333333333333333300000000
3. The IV is 00000000000000000000000000000000

Packaging content for Marlin

A tutorial on packaging content for Marlin DRM is available here

Lesson 4 - DRM License redemption workflow

The license acquisition workflow is captured in the sequence diagram below:



The standard license acquisition workflow with ExpressPlay involves the following steps:

1. The application signals to the Over the top (OTT) service that it needs a license for a specific content element
2. The OTT service verifies that the request came from an authorized user and determines whether the user is allowed to view the content, if not it will reject the license request
3. The OTT service calls the ExpressPlay service, passing the content decryption key and the desired permissions

4. ExpressPlay returns a token, which is an encrypted BLOB that contains the information required to construct a license
5. The OTT service converts the ExpressPlay Token into a license acquisition URL that is returned to the application. The license acquisition URL will point to the relevant ExpressPlay licensing service and specify the token in a query string parameter (for example: https://wv.service.expressplay.com/hms/wv/rights/?ExpressPlayToken=<Token>)
6. The application initializes the player with the LA_URL and starts playback
7. The player retrieves the encrypted content from the CDM and passes it to the media stack for decryption
8. The player retrieves the DRM license from ExpressPlay

Note: In addition the workflow mentioned above, ExpressPlay supports an alternate deployment model, known as the License-Proxy model in which the service returns a license and not an authorization token to the client. There are some advantages to this approach, from a security and implementation standpoint, please see further details here

Token Lifetime

By default, all tokens are valid for 30 days, however, an expiration time can be specified when a token is generated.

Note: There is no charge for tokens and you will only be charged for the licenses redeemed, hence there is no reason to re-use the same token and in fact, it's highly advised to use short-lived tokens (e.g. 10 seconds) to mitigate against illegitimate re-use of tokens.

The ExpressPlay REST API

The ExpressPlay REST API is used to generate the DRM tokens and retrieve logs on token redemptions, the documentation for this API is available here.

Note: The documentation specifies URLs for both Test and Production environments. The Test environment is in the process of being deprecated for all DRMs except Marlin.

Generating ExpressPlay Tokens

Below are some sample curl commands for retrieving Marlin, Widevine, PlayReady, FairPlay tokens for the content packaged in step 3. The sample commands assume:

- That an environment variable called API_KEY has been initialized with a valid ExpressPlay API key, as demonstrated below

```
export API_KEY=971450948,6805ae268f7149609e4eb9ba11644dbf
```

- The use of Bash shell, which requires double quotes around the API_KEY parameter (other shells likely have different conventions for referring to environment variables).

With the exception of PlayReady and Marlin Broadband, the response of these API calls is the full license acquisition URL (LA_URL) , that can be directly passed to the player (more on PlayReady and player configuration below)

FairPlay Streaming

Below is a sample command to create a FairPlay Streaming token. The documentation for the FairPlay token generation API is available here

```
curl -X GET
'https://fp-gen.service.expressplay.com/hms/fp/token?customerAuthen
ticator='"$API_KEY"'&errorFormat=json&iv=00000000000000000000000000
0000000&contentKey=7f45050639469c1fb2a85db718c28ed7&useHttps=tru
e'
```

Note: When redeeming a FairPlay license it's required to specify an HTTP header of 'Content-Type' that equals 'application/octet-stream'.

The API response can be used as a license acquisition URL in the player configuration:

```
https://fp.service.expressplay.com/hms/fp/rights/?ExpressPlayToken=
BQAqVaU0KbYAJDI2NzUwMjgyLTI2M2ItNGRlZi1iZGQzLTZiNzdiNjM4ZDNhZAAAAFA
kjtE3snaQrvpo6DsC1HsNPOxXPWXzC7leyJ5Fwf9fU7Acg7hgIuJQdfVqhKdb78yvpX
kKdDIUjmld-fHhg-1Dami0M2IVA0Wyogb5djS0xwbr7s6MBB4679ywy7O_0CFiHdLR
```

Widevine Modular Token Generation

Below is a sample command to create a Widevine token. The documentation for the Widevine token generation API is available here

```
curl -X GET
'https://wv-gen.service.expressplay.com/hms/wv/token?customerAuthen
ticator='"$API_KEY"'&errorFormat=json&kid=33333333333333333333333330
0000000&contentKey=7f45050639469c1fb2a85db718c28ed7&useHttps=true'
```

The API response can be used as a license acquisition URL in the player configuration:


```
https://wv.service.expressplay.com/hms/wv/rights/?ExpressPlayToken=
BQAqVaU0KaQAJDI2NzUwMjgyLTI2M2ItNGRlZi1iZGQzLTZiNzdiNjM4ZDNhZAAAAGA
Q3edV8Fe7HjVKKlqrQh1BILjUi-wz5n5nFnAUP0QiogJ_oBS7AaFGu2VUh9VDzzx8RT
ZlRglMJL6NojULe-eRKVBeueYKix5ogWXxefr-FkRmZUqcNDXfq2Mzh5FXDEkdLdVIH
E7vEiVeiEZ7i9Qdbci-2Q
```


PlayReady DRM Token Generation

Below is a sample command to create a PlayReady token. The documentation for the PlayReady token generation API is available here

```
curl -X GET
'https://pr-gen.service.expressplay.com/hms/pr/token?customerAuthen
ticator='"$API_KEY"'&errorFormat=json&kid=33333333333333333333333330
0000000&contentKey=7f45050639469c1fb2a85db718c28ed7&rightsType=BuyT
oOwn&uncompressedDigitalVideoOPL=0&compressedDigitalVideoOPL=0&unco
mpressedDigitalAudioOPL=0&compressedDigitalAudioOPL=0&analogVideoOP
L=0&useHttps=true'
```

The PlayReady token generation API does not return a complete license acquisition URL, but rather a JSON object with “licenseAcquisitionUrl” and “token” fields.

The licenseAcquisitionUrl field is the URL at which the PlayReady license server is reachable. Depending on the client platform on which your media application is running, there are different ways to make use of the “token” value to trigger the PlayReady license acquisition.
Further details are available here

HTML5 players typically require a single license acquisition URL, so the two fields need to be concatenated to form the LA_URL , for example the response below:


```
{"licenseAcquisitionUrl":"https://pr.service.expressplay.com/playrea
dy/RightsManager.asmx","token":"BQAqVaU0N20AJDI2NzUwMjgyLTI2M2ItNGR
lZi1iZGQzLTZiNzdiNjM4ZDNhZAAAAHC9L4jCbVj7VMucJlGBc_vb6WC9-ZcNVrLJba
3-CaXOvKBnNMGItluaaUOiWKQzKhO0tRgHSKwDSjSQhqjx-B0151fEX4BsZMXpcehTQ
U9fVFQp0W8OlwV1kG1wvfmwfGXctbMhIsq_YEJx6BLbMFgPPAUzSX_jQ0KFfvE2iQG5
r_p6WoA"}
```


Should be converted to the following LA_URL before it is provided to the player.

```
https://pr.service.expressplay.com/playready/RightsManager.asmx?Exp
ressPlayToken=BQAqVaU0N20AJDI2NzUwMjgyLTI2M2ItNGRlZi1iZGQzLTZiNzdiN
jM4ZDNhZAAAAHC9L4jCbVj7VMucJlGBc_vb6WC9-ZcNVrLJba3-CaXOvKBnNMGItlua
aUOiWKQzKhO0tRgHSKwDSjSQhqjx-B0151fEX4BsZMXpcehTQU9fVFQp0W8OlwV1kG1
wvfmwfGXctbMhIsq_YEJx6BLbMFgPPAUzSX_jQ0KFfvE2iQG5r_p6WoA
```

ExpressPlay PlayReady Service supports some other popular methods for specifying the token. (eg. through an HTTP Header, etc.). The various options are described here.

Marlin MS3 and Marlin Broadband Token Generation

A tutorial on generating Marlin Tokens is available here

Lesson 5 - The ExpressPlay Audit Log

The history of the past 3 months of license acquisition events is available in the ExpressPlay audit log.

It's possible to query the log through the following API's, click on each of the link below for further details

- getRecord
- getrecordbytime
- getarchivedlog

Below is a sample call to retrieve the 32 most recent WV transactions

```
curl -X GET \
'https://api.service.expressplay.com/cmiapi/getrecord?customerAuth
enticator='"$API_KEY"'&type=wv'
```

Note that it's possible to retrieve the record for a specific record by specifying a correlation value when requesting a token (see cookie parameter in the DRM token generation API call)

Lesson 6 - Content Playback

ExpressPlay has been tested and deployed with all the mainstream A/V players, such as:

- HTML5 Players
    - BitMovin
    - TheoPlayer

    - JWPlayer
    - ShakaPlayer
    - VideoJS
    - DashJS
- AVPlayer on iOS
- ExoPlayer on Android
- As well as third party native players such as NexPlayer, VisualOn, and Labgency

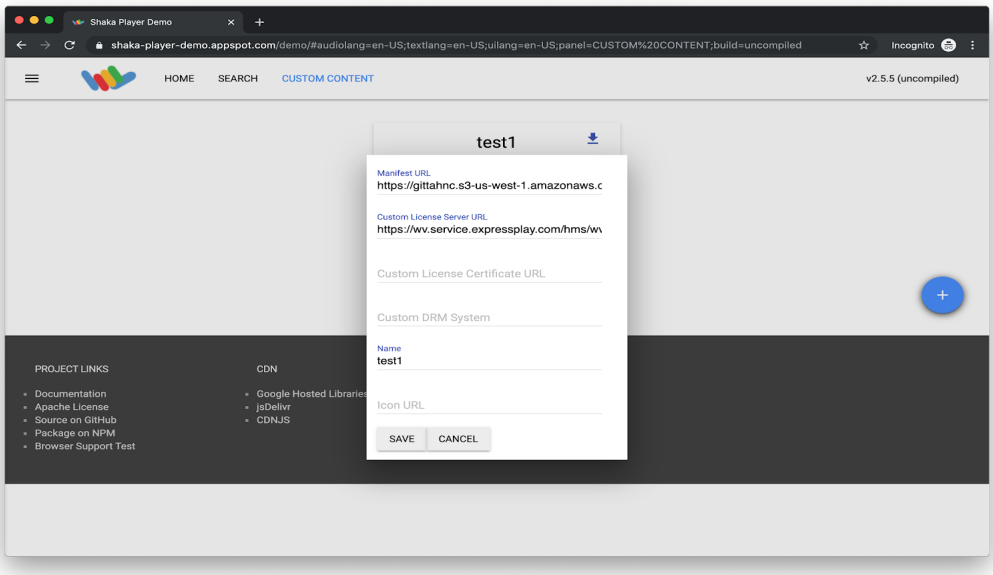
Shaka Player online DRM demo

An online demo of Shaka Player is available at https://shaka-player-demo.appspot.com/

The “Custom” section of the demo allows for playback for Widevine protected content on Chrome and PlayReady protected content on Microsoft browsers. All that is required is the ExpressPlay token for a Widevine license and a DASH manifest for some protected content.

The manifest below corresponds to content protected with the parameters matching the Widevine token generated in section 4 of this guide, paste it in the Manifest URL field of Custom Content form in the Shaka Player demo along with the Widevine token generated in section 4.

```
https://expressplay.s3.amazonaws.com/sample%20content/Sample%20A/DASH/stream.mpd
```



Shaka Player online demo

HTML5 Playback

Playback of protected content is typically straightforward, requiring a configuration of the player with the license acquisition URL returned by ExpressPlay.

The example below demonstrates configuration of JWPlayer player, to play some FairPlay protected content:

```
jwplayer("myElement").setup({"sources": [{
        "default": false,
        "file":
"https://expressplay.s3.amazonaws.com/sample%20content/Sample%20A/HLS/master.m3u8",
        "type": "hls",
        "drm": {
        "fairplay": {
            "processSpcUrl":
"https://fp.service.expressplay.com/hms/fp/rights/?ExpressPlayToken=BQAqVaU0KZQAJDI2
NzUwMjgyLTI2M2ItNGRlZi1iZGQzLTZiNzdiNjM4ZDNhZAAAAFAR8SpiNJn3Rw0nV5D_9ljno6OeWI2ZBXOp
Cfe-dcqfAiwSeOP2n9gwN5doRMEkdr422IMxrVBB4sr-YODzd1VdWrNN1vZdmbbtK6lKbt_E876DANzsdpuv
mwCMkA1r_DCR47ff",
                "certificateUrl":
"https://gittahnc.s3-us-west-1.amazonaws.com/temp/pc_fairplay/fairplay.cer",
                licenseRequestHeaders: [ { name: 'Content-type', value:
'application/octet-stream' } ] }},
        "label": "0"
    }}]});
```

Note: When redeeming a FairPlay license it's required to specify an HTTP header of 'Content-Type' that equals 'application/octet-stream'.

Note: A common issue with Web playback is CORS support and configuration of the server the content is being downloaded from

Playback using ExoPlayer

A sample of playback of Widvine protected content using ExoPlayer is available here

Playback using AVPlayer

A sample of playback of FairPlay protected content using AVPlayer is available here

Note: When redeeming a FairPlay license it's required to specify an HTTP header of 'Content-Type' that equals 'application/octet-stream'.

Playback of Marlin protected content

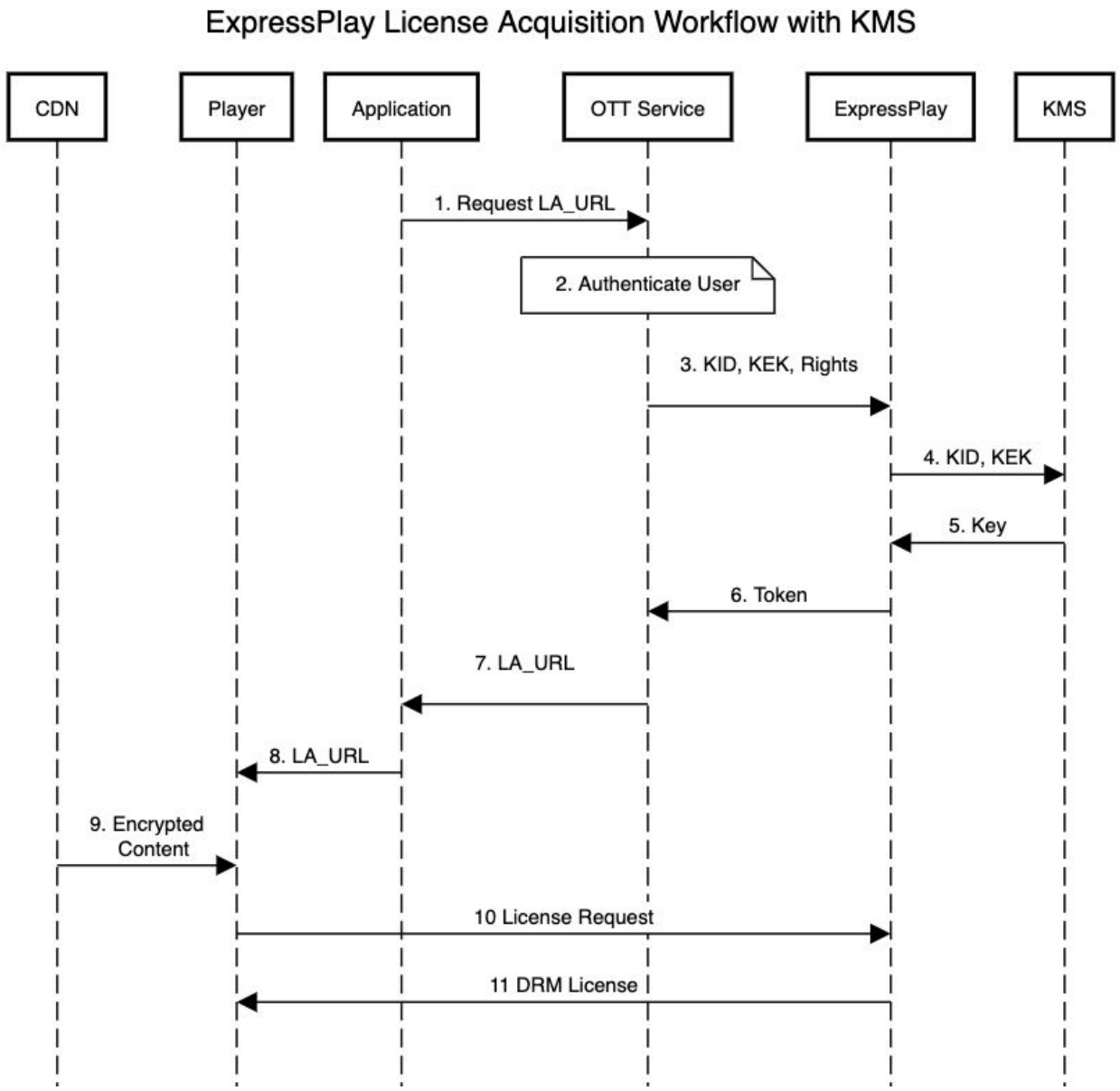
Playback of Marlin protected content requires the Marlin SDK, tutorials on the use of the SDK are available here and here

Lesson 7 - The Key Management Service (KMS)

ExpressPlay offers a key management service at no additional costs, the service enables secure generation and management of content keys and is fully integrated with the ExpressPlay token generation API.

When generating a DRM token a key stored in the KMS is referenced by its Key Identifier (KID), for example, the request below will generate a FairPlay token corresponding to the key stored with the KID of 33333333333333333333333300000000 (Note the key with the KID must have first been added to the KMS)

```
curl -X GET
'https://fp-gen.service.expressplay.com/hms/fp/token?customerAuthen
ticator='"$API_KEY"'&errorFormat=json&kid=33333333333333333333333330
0000000&kek=11111111111111111111111111111111&iv=0000000000000000000
0000000000000&useHttps=true'
```



Third-Party Packaging Services

The ExpressPlay KMS has been pre-integrated with the leading packaging services, including AWS Elemental Media Package, BitMovin, Wowza, Fastly's on-the-fly packaging service.

* A tutorial on using AWS Elemental services with ExpressPlay is available here
* ExpressPlay provides a plug-in for the Wowza streaming server, configuration details available for the plug-in are available here for DASH content and here for HLS content
* Tutorials on using the BitMovin encoder API are available here and here
* Details on the Fastly's on-the-fly packaging service are available here

KMS Sample commands

Following are some sample commands to retrieve, generate and store/update keys in the KMS. The sample commands assume:

- That an environment variable called API_KEY has been initialized with a valid ExpressPlay API key
- The use of Bash shell, which requires double quotes around the API_KEY parameter (other shells likely have different conventions for referring to environment variables)

Operations on the KMS typically involve a key, a key identifier (KID) and a key used to encrypt the stored encryption keys, (Key Encryption Key known as the KEK). All of which are 16 bytes in length and are formatted as a 32-byte hex string.

Keys are populated in the KMS either by requesting the KMS to generate a key or specified and stored directly as demonstrated in the samples below.

Further details on the KMS API and cookbooks are available [here]

Retrieving a Content Key

```
curl -X GET \
'https://api.service.expressplay.com/keystore/keys/3333333333333333
3333333300000000?customerAuthenticator='"$API_KEY"'&kek=11111111111
11111111111111111111'
```

This will yield a response similar to the one below the key value is in the “k” field of the response

```
{
  "kid":"33333333333333333333333300000000","
  "kekId":"#1.952241B0259652B68A8CAE2BE819DFA3390CDB35","
  "ek":"2B62A42346234C4C34C9A824BF4952091F7C95AB33E4A313","
  "k":"7F45050639469C1FB2A85DB718C28ED7","
  "lastUpdate":"2019-10-11T01:54:20Z"
}
```

Generating a Content Key

```
curl -X POST
'https://api.service.expressplay.com/keystore/keys?customerAuthenti
cator='"$API_KEY"'&kek=11111111111111111111111111111111'
```

Storing a content key under a specific Key identifier

```
curl -X POST \
'https://api.service.expressplay.com/keystore/keys?customerAuthenti
cator='"$API_KEY"'&kek=11111111111111111111111111111111' \
-H 'Content-Type: application/json' \
-d '{
       "kid": "33333333333333333333333300000000",
        "k":   "7F45050639469C1FB2A85DB718C28ED7"
}'
```

Updating a content key

```
curl -X PUT \
'https://api.service.expressplay.com/keystore/keys/3333333333333333
3333333300000000?customerAuthenticator='"$API_KEY"'&kek=11111111111
11111111111111111111' \
-H 'Content-Type: application/json' \
-d '{
        "k":   "7F45050639469C1FB2A85DB718C28ED7"
}'
```

Implementing Key Rotation

Key Rotation refers to periodic changes in the key used to encrypt a stream, typically a live stream. There are many ways this can be accomplished but the most straight forward is for the encoder to store all the keys related to the stream under the same KID prefix.

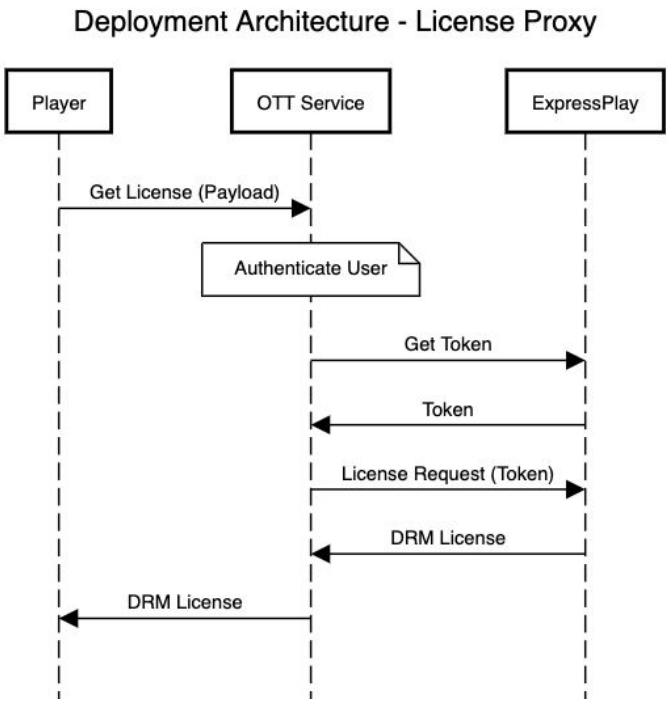
For example, the keys below correspond to a 24-hour feed with a 6 hour key rotation period.

During playback, the player determines which 6 hour period is being played (for example by extracting it from the segment name, the media manifest, or relative location in the feed) and requests the license matching the appropriate key from the service.

Period	Key ID
0-6 hours	112233445566778899AABBCCDD0000
6-12 hours	112233445566778899AABBCCDD0001
12-18 hours	112233445566778899AABBCCDD0002
18-24 hours	112233445566778899AABBCCDD0003

Lesson 8 - License Proxy Deployment Architecture

An alternate deployment architecture supported by ExpressPlay is the license proxy deployment, which enables the player to directly retrieve a DRM license from the ExpressPlay service.



This license delivery offers the following benefits

- Reduced latency - As opposed to the token license delivery model, which requires two round trips to ExpressPlay, this approach, if optimized requires only 1 call on average.
- Mitigation of replay attacks - The license returned by ExpressPlay is bound to the particular session the license was obtained for.
- The OTT service has an opportunity to implement additional checks before the license is returned to the device (e.g. geo-blocking) before returning the license
- Simpler client-side logic
    - As the player will always retrieve the license from a known endpoint, the player set up is simpler because it doesn't require prefetching of the token
    - Any errors in retrieving the token/license can be handled on the server end

Note: The License-Proxy model does not work with Marlin DRM

Performance considerations

- The application can derive a license acquisition URL based on the license type and a content identifier (e.g. www.ottservice.com/license/wv?id=ContentId) eliminating the need for a round trip to the service to obtain the URL
- The service can generate long-lived tokens (up to 30 days) and cache them for subsequent license retrieval requests

Payload and HTTP headers

The request from the client will include a payload that must be passed to the license server along with the token. The request to the license server should include

1. The HTTP Content-Type header as outlined in the table below:

DRM System	**Content-Type Header**
Apple FairPlay	application/octet-stream
Google Widevine	application/octet-stream
Microsoft PlayReady	text/xml

2. PlayReady clients may also pass SOAP HTTP headers, which should be forwarded to the license server